서울 , 어디서 놀고 어디서 벌까 ?

공공데이터를 방문 · 창업 의사결정 서비스로 연결하다

분석 근거와 실제 구현을
하나의 흐름으로 설명합니다 .

84,675
분석 관측값

1,577
서울 상권

62
생활밀착업종

23
핵심 코드 발췌

목적이 달라지면 같은 상권의 해석도 달라진다

방문 목적과 창업 목적에 맞는 지표를 분리했습니다 .

CONSUMER

어디서 놀까 ?

2030 비중과 주말 · 야간 활동량을 함께 비교합니다 .

BUSINESS

어디서 벌까 ?

업종별 매출과 수요 · 폐업점포 비율을 함께 봅니다 .

ENGINEERING

어떻게 연결할까 ?

공통 데이터마트 위에 추천 규칙 · 매출 추정 · 웹 화면을 구성합니다 .

추천 점수는 규칙 기반 상대평가 , 머신러닝 출력은 예상 월매출입니다 .

원천 데이터에서 화면까지 이어지는 구현 구조

데이터 처리 · 추천 계산 · 모델 학습 · 서비스 실행의 책임을 나눴습니다 .

RAW DATA

매출 · 점포
유동인구 · 영역

→

DATA MART

키 정합성 · 결측 처리
공통 파생변수

→

RULE / MODEL

백분위 추천
Random Forest

→

STREAMLIT

업종별 TOP10
핫플 TOP20 · 지도

전처리 · 학습은 오프라인 스크립트에서 실행하고 ,
서비스는 저장된 CSV 와 모델을 읽어 화면과 추론을 제공합니다 .

무엇을 수집했고, 무엇을 실제로 사용했나

출처 : 서울 열린데이터광장 · 서울시 상권분석서비스 / 기존 발표 제공기관 표기 : 서울신용보증재단

원천 데이터	원본 행 × 열	보관 범위	현재 사용 목적
추정매출	85,732 × 55	2025 Q1–Q4	매출 · 객단가 · 모델 타깃
점포	304,775 × 14	2025 Q1–Q4	점포 수 · 폐업점포 비율
유동인구	36,281 × 27	2021 Q1–2026 Q2	분석에는 2025 년 결합
상권영역	1,650 × 11	시점 컬럼 없음	좌표 · 자치구 · 행정동

CSV 보관과 읽기 코드는 확인했습니다. 실제 수집일 · 개별 다운로드 URL·API 수집 여부는 미확인입니다.
data.seoul.go.kr / golmok.seoul.go.kr

보관 데이터의 범위와 분석 결과의 시점을 구분

파일이 있다고 모두 추천 · 예측에 반영한 것은 아닙니다 .

ACTIVE

2025 년 분석

정제 데이터 84,675 행
Q1 21,170 / Q2 21,186
Q3 21,256 / Q4 21,063

ARCHIVE

2024 년 자료

매출 87,179 행
점포 306,889 행
현재 처리 코드에는 미반영

NOT USED

집객시설

34,716 행 × 25 열
결측 셀 563,750 개
현재 모델 · 추천 입력 제외

2026 년 원본이 일부 있어도 현재 서비스의 최신 핫플 결과는 2025 Q4 기준입니다 .

CSV 읽기와 기준 시점 분리

서로 다른 파일을 같은 시점으로 연결하기 위한 준비

01_check_data.py · L12–L21

```
sales = pd.read_csv(
    BASE_DIR / 'data/raw/sales_2025.csv',
    encoding="cp949"
)
sales[" 기준 _ 년 _ 코드 "] = sales[" 기준 _ 년분기 _ 코드 "] // 10
sales[" 기준 _ 분기 _ 코드 "] = sales[" 기준 _ 년분기 _ 코드 "] % 10
```

실제 코드 01 / 23 · 원본 코드 열기 · 01_check_data.py

무엇을 구현했나

cp949 로 읽고 , 연분기 코드를 정수 나눗셈과 나머지로 분리

사용자에게 연결되는 결과

20251 → 연도 2025, 분기 1

개발 관점의 설계 판단

파일 인코딩과 시점을 먼저 표준화해 이후 결합 기준을 일관되게 유지했습니다 .

매출과 점포를 동일한 키로 병합

상권명이 같아도 업종 · 분기가 다르면 다른 관측값

01_check_data.py · L123–L134

```
mart = pd.merge(
    sales,
    store,
    on=[
        "기준_년_코드",
        "기준_분기_코드",
        "상권_코드",
        "서비스_업종_코드"
    ],
    how="left",
    validate="many_to_one",
)
```

실제 코드 02 / 23 · 원본 코드 열기 · 01_check_data.py

무엇을 구현했나

연도 · 분기 · 상권 코드 · 업종 코드로 left merge

사용자에게 연결되는 결과

매출의 행을 유지한 채 점포 정보를 연결

개발 관점의 설계 판단

코드 기반 키와 many_to_one 검증으로 이름 중복 · 우측 키 중복에 따른 행 증가를 방지합니다 .

인구를 상권 · 분기에 맞춰 연결

인구에는 업종 키가 없으므로 병합 단위를 달리 적용

01_check_data.py · L302–L312

```
final_mart = pd.merge(
    mart,
    population_merge,
    on=[
        "기준_년_코드",
        "기준_분기_코드",
        "상권_코드"
    ],
    how="left",
    validate="many_to_one",
)
```

실제 코드 05 / 23 · 원본 코드 열기 · 01_check_data.py

무엇을 구현했나

연도 · 분기 · 상권 코드로 연결하고 우측 키 유일성 검증

사용자에게 연결되는 결과

업종별 매출 행에 같은 상권의 인구 정보가 반복됨

개발 관점의 설계 판단

인구에는 업종 키가 없다는 데이터 구조를 반영했습니다 . 업종 행에 반복된 인구는 다시 합산하지 않습니다 .

분석에 필요한 핵심 결측만 제외

의미 있는 비교에 필요한 최소 입력을 확보

02_clean_data.py · L38–L44

```
clean_df = df.dropna(
    subset=[
        "점포당_월매출",
        "총_유동인구_수",
        "2030비율"
    ]
).copy()
```

실제 코드 06 / 23 · 원본 코드 열기 · 02_clean_data.py

무엇을 구현했나

월매출 · 총 유동인구 · 2030 비율 중 하나라도 없으면 제외

사용자에게 연결되는 결과

85,732 행 → 84,675 행

핵심 결측 1,057 행 제외

개발 관점의 설계 판단

임의의 0 채움 대신 분석 필수 컬럼을 기준으로 제외했습니다. 정제 결과 전체 결측 · 행 중복은 0 입니다.

매출과 폐업점포 비율 만들기

단순 총매출을 점포 규모와 거래 건수로 보정

01_check_data.py · L149–L173

```
mart['객단가'] = (
    mart['분기당_매출_금액']
    / mart['분기당_매출_건수'].replace(0, pd.NA)
)
mart['점포당_분기매출'] = (
    mart['분기당_매출_금액']
    / mart['점포_수'].replace(0, pd.NA)
)
mart['점포당_월매출'] = (
    mart['점포당_분기매출'] / 3
)
mart['폐업비율'] = (
    mart['폐업_점포_수']
    / mart['점포_수'].replace(0, pd.NA)
    * 100
```

실제 코드 03 / 23 · 원본 코드 열기 · 01_check_data.py

무엇을 구현했나

분모 0을 NA로 바꾼 뒤 객단가·월매출·폐업비율 계산

사용자에게 연결되는 결과

상권·업종별 비교와 모델 타깃에 쓰는 파생변수

개발 관점의 설계 판단

분모 0을 NA로 바꿔 잘못된 비율 생성을 막습니다. 분기 ÷3 해석은 원천 메타데이터 대조가 필요합니다.

2030 방문층 비중 계산

총량이 비슷해도 연령 구성이 다른 상권을 구분

01_check_data.py · L241–L252

```
population["2030_ 유동인구 _ 수 "] = (
    population[" 연령대 _20_ 유동인구 _ 수 "]
    + population[" 연령대 _30_ 유동인구 _ 수 "]
)
population["2030 비율 "] = (
    population["2030_ 유동인구 _ 수 "]
    / population[" 총 _ 유동인구 _ 수 "].replace(0, pd.NA)
    * 100
)
```

실제 코드 04 / 23 · 원본 코드 열기 · 01_check_data.py

무엇을 구현했나

20 대와 30 대 인구를 합산하고 전체 인구로 나눠 백분율화

사용자에게 연결되는 결과

추천점수와 Random Forest 입력에 공통 사용

개발 관점의 설계 판단

사람이 많은 곳과 젊은층 비중이 높은 곳을 분리했습니다 . 같은 파생변수를 추천과 모델에 재사용합니다 .

백분위와 폐업 안정성 방향 반전

원 · 명 ·% 단위를 같은 상대점수로 비교

04_better_score.py · L23–L57

```python
def percentile_score(series):
    return series.rank(
        pct=True
    ) * 100
df[" 매출점수 _v2"] = percentile_score(
    df[" 점포당 _ 월매출 "]
)
df[" 유동인구점수 _v2"] = percentile_score(
    df[" 총 _ 유동인구 _ 수 "]
)
df["2030 점수 _v2"] = percentile_score(
    df["2030 비율 "]
)
df[" 폐업안정점수 _v2"] = (
    100
    - percentile_score(
        df[" 폐업비율 "]
```

실제 코드 07 / 23 · 원본 코드 열기 · 04_better_score.py

무엇을 구현했나

rank(pct=True) × 100, 폐업비율만 100 에서 차감

사용자에게 연결되는 결과

매출 · 인구 · 젊은층은 클수록 , 폐업비율은 작을수록 유리

개발 관점의 설계 판단

서로 다른 단위와 좋고 나쁜 방향을 통일합니다 . 점수의 비교 범위는 전체 분기 · 업종 관측 행입니다 .

창업추천점수 가중합

창업 목적에 맞춰 수익성과 수요 · 안정성을 결합

04_better_score.py · L61–L80

```
df[" 창업추천점수 _v2"] = (
    df[" 매출점수 _v2"] * 0.40
    + df[" 유동인구점수 _v2"] * 0.25
    + df["2030 점수 _v2"] * 0.20
    + df[" 폐업안정점수 _v2"] * 0.15
)
recommend = df[
    df[" 점포 _ 수 "] >= 3
].copy()
ranking = recommend.sort_values(
    " 창업추천점수 _v2",
    ascending=False
)
```

실제 코드 08 / 23 · 원본 코드 열기 · 04_better_score.py

무엇을 구현했나

매출 40%, 인구 25%, 2030 20%, 안정성 15%

사용자에게 연결되는 결과

매출 40% · 유동 25%

2030 20% · 안정성 15%

개발 관점의 설계 판단

추천 규칙을 모델 추론과 분리했습니다 . 04 의 저장값은 전체 df 이며 , 최종 서비스 필터는 06 에서 다시 적용합니다 .

서비스 후보 필터와 기간 평균

분기별 단기 변동을 상권 · 업종별 요약값으로 제공

06_final_startup_ranking.py · L21–L45

```
df = df[df[" 점포 _ 수 "] >= 3].copy()
startup = (
    df.groupby(
        [" 상권 _ 코드 ", " 상권 _ 코드 _ 명 ",
         " 서비스 _ 업종 _ 코드 ", " 서비스 _ 업종 _ 코드 _ 명 "],
        as_index=False
    ).agg(
        평균 _ 점포 _ 수 =(" 점포 _ 수 ", "mean"),
        평균 _ 점포당 _ 월매출 =(" 점포당 _ 월매출 ", "mean"),
        평균 _ 총 _ 유동인구 =(" 총 _ 유동인구 _ 수 ", "mean"),
        평균 _2030 비율 =("2030 비율 ", "mean"),
        평균 _ 폐업비율 =(" 폐업비율 ", "mean"),
        평균 _ 창업추천점수 =(" 창업추천점수 _v2", "mean")
    )
```

실제 코드 09 / 23 · 원본 코드 열기 · 06_final_startup_ranking.py

무엇을 구현했나

점포 수 ≥ 3 필터 후 groupby 와 mean 으로 집계

사용자에게 연결되는 결과

서비스 후보 20,933 개

상권 · 업종별 기간 평균

개발 관점의 설계 판단

점포 수 ≥ 3 을 통과한 분기만 평균에 포함합니다 . 표본 조건을 적용하되 관측 분기 수 공개는 개선 과제입니다 .

가구 업종의 추천 결과는 실제 화면과 연결된다

전체 업종 통합 순위와 구분해 , 사용자가 선택한 업종 안에서 비교합니다 .



근거 : startup_ranking.csv · 임대료 · 원가가 없어 추천점수는 수익성 보장이 아닙니다 .

핫플의 분석 단위 중복 제거

동일 상권의 인구가 업종마다 반복되는 문제를 해결

05_hotplace_score.py · L23–L36

```
hot = df[
    [
        "기준_년_코드",
        "기준_분기_코드",
        "상권_코드",
        "상권_코드_명",
        "총_유동인구_수",
        "2030 비율",
        "시간대_17_21_유동인구_수",
        "시간대_21_24_유동인구_수",
        "토요일_유동인구_수",
        "일요일_유동인구_수"
    ]
].drop_duplicates()
```

실제 코드 10 / 23 · 원본 코드 열기 · 05_hotplace_score.py

무엇을 구현했나

업종 컬럼 없이 상권 · 분기 · 인구 지표만 선택하고 drop_duplicates

사용자에게 연결되는 결과

핫플 계산 대상 6,271 개 상권 · 분기 관측값

개발 관점의 설계 판단

업종 행마다 반복된 인구를 제거합니다 . 상권 · 분기별 관측값을 유지해 핫플 총량 중복을 줄였습니다 .

주말 · 야간 활동량 계산

언제 붐비는지 방문 목적에 맞는 지표로 요약

05_hotplace_score.py · L43–L53

```
hot[" 주말 _ 유동인구 _ 수 "] = (
    hot[" 토요일 _ 유동인구 _ 수 "]
    + hot[" 일요일 _ 유동인구 _ 수 "]
)
hot[" 야간 _ 유동인구 _ 수 "] = (
    hot[" 시간대 _17_21_ 유동인구 _ 수 "]
    + hot[" 시간대 _21_24_ 유동인구 _ 수 "]
)
```

실제 코드 11 / 23 · 원본 코드 열기 · 05_hotplace_score.py

무엇을 구현했나

토 + 일을 주말로 , 17~21 시 +21~24 시를 야간으로 정의

사용자에게 연결되는 결과

주말 = 토 + 일

야간 = 17~24 시

개발 관점의 설계 판단

시간대의 정의를 코드로 명확히 했습니다 . 현재 지표는 비중이 아닌 인구 총량이므로 해석에 주의합니다 .

핫플점수와 최신 분기 추출

전체 활동성에 젊은층 · 주말 · 야간 특성을 반영

05_hotplace_score.py · L80–L99

```
hot[" 핫플점수 "] = (
    hot[" 전체유동점수 "] * 0.35
    + hot["2030 핫플점수 "] * 0.30
    + hot[" 주말점수 "] * 0.20
    + hot[" 야간점수 "] * 0.15
)
latest_year = hot[" 기준 _ 년 _ 코드 "].max()
latest_quarter = hot[
    hot[" 기준 _ 년 _ 코드 "] == latest_year
][" 기준 _ 분기 _ 코드 "].max()
latest = hot[
    (hot[" 기준 _ 년 _ 코드 "] == latest_year)
    &
    (hot[" 기준 _ 분기 _ 코드 "] == latest_quarter)
].copy()
```

실제 코드 12 / 23 · 원본 코드 열기 · 05_hotplace_score.py

무엇을 구현했나

백분위 가중합 후 max 연도와 해당 연도의 max 분기 선택

사용자에게 연결되는 결과

유동 35% · 2030 30%

주말 20% · 야간 15%

개발 관점의 설계 판단

백분위를 만든 뒤 최신 분기만 선택합니다 . 총량 · 주말 · 야간의 상관성에 따른 영향 중첩은 검증 과제입니다 .

젊은층 · 주말 · 야간 활동성을 함께 반영한 순위

2025 년 4 분기 핫플 결과 · 실시간 혼잡도 순위와 구분합니다 .



근거 : hotplace_score.csv · 상권 코드가 다른 인접 상권은 별도 후보입니다 .

상권 좌표를 지도 위도 · 경도로 변환

국내 평면좌표를 웹 지도의 지리좌표로 맞춤

07_add_location.py · L37–L51

```
transformer = Transformer.from_crs(
    "EPSG:5181",
    "EPSG:4326",
    always_xy=True
)
longitude, latitude = transformer.transform(
    area[" 엑스좌표 _ 값 "].values,
    area[" 와이좌표 _ 값 "].values
)
area[" 경도 "] = longitude
area[" 위도 "] = latitude
```

실제 코드 13 / 23 · 원본 코드 열기 · 07_add_location.py

무엇을 구현했나

EPSG:5181 → EPSG:4326, always_xy=True

사용자에게 연결되는 결과

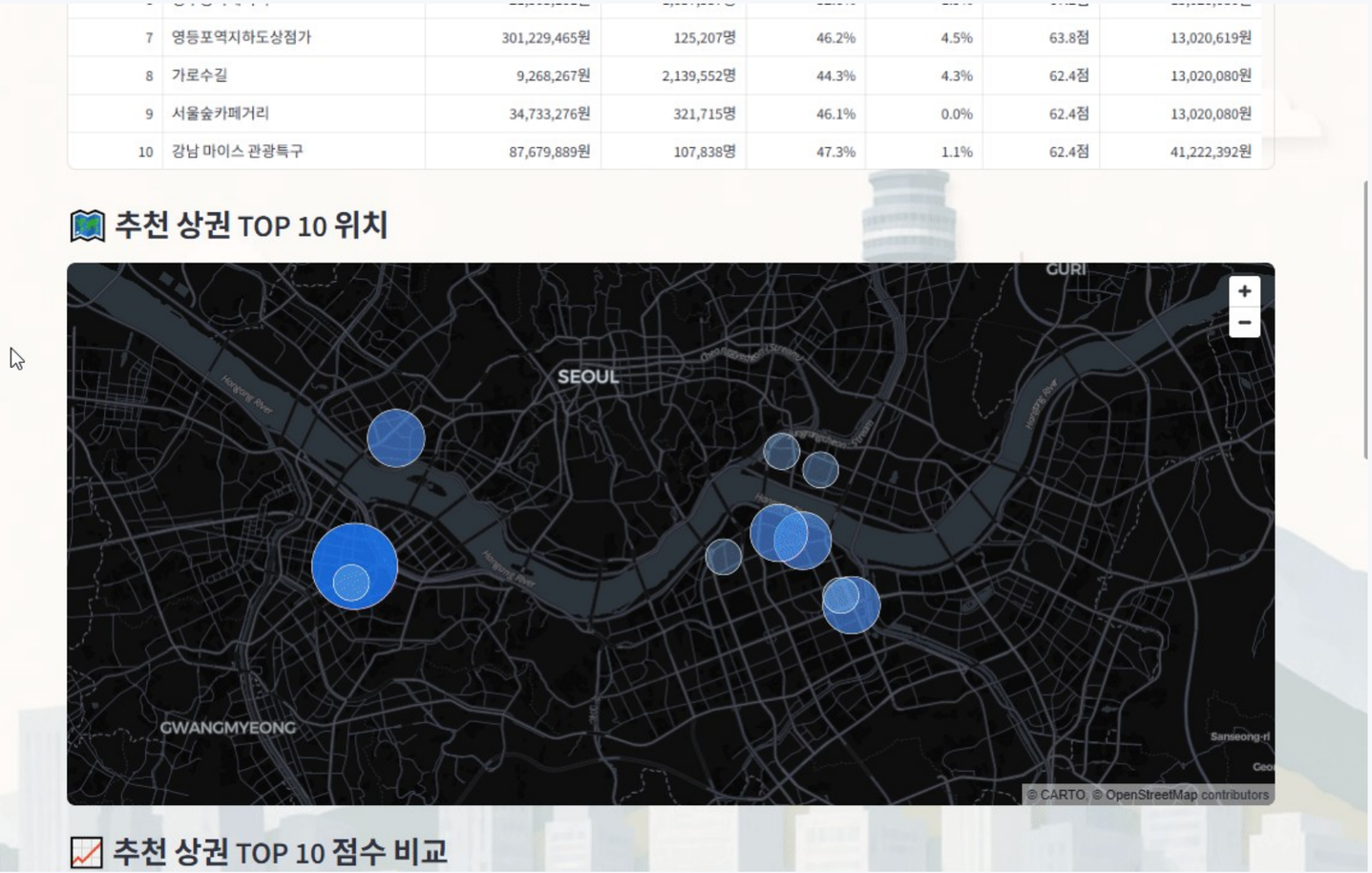
변환값을 경도 · 위도 컬럼에 저장

개발 관점의 설계 판단

좌표체계를 바꾼 뒤 상권 코드로 위치를 연결합니다 . 지도용 저장 테이블의 좌표 결측은 0 건입니다 .

분석 순위를 지도에서 비교할 수 있도록 구현

추천 결과에 위도 · 경도를 결합하고 , 순위에 따라 마커 크기와 색상을 달리합니다 .



코드로 확인되는 설계

상권 코드별 위치 중복 제거

결측 좌표를 지도에서 제외

공통 지도 함수로 두 모드 재사용

순위별 마커 · 툴팁 제공

모델 입력과 예측 대상을 명시

머신러닝이 어떤 정보로 무엇을 예측하는지 정의

12_final_ai_model.py · L27–L39

```
numeric_features = [
    "총_유동인구_수",
    "2030 비율",
    "점포_수",
    "폐업비율"
]
categorical_features = [
    "서비스_업종_코드_명",
    "상권_코드_명"
]
target = "점포당_월매출"
```

실제 코드 14 / 23 · 원본 코드 열기 · 12_final_ai_model.py

무엇을 구현했나

수치형 4 개 + 범주형 2 개 → 점포당 월매출

사용자에게 연결되는 결과

추천 순위와 별도로 예상 월매출을 제공

개발 관점의 설계 판단

입력과 타깃을 명시해 추천 규칙과 모델의 역할을 구분합니다. 월매출 평균 추정이며 개별 점포 이익 예측은 아닙니다.

이후 분기를 분리하고 학습 이상치 처리

평가 분기를 학습과 이상치 기준 산출에서 분리

11_time_test_model.py · L69–L88

```
train_df = model_df[
    model_df['기준_분기_코드'].isin([1, 2, 3])
].copy()
test_df = model_df[
    model_df['기준_분기_코드'] == 4
].copy()
upper_limit = train_df[target].quantile(0.99)
train_df = train_df[train_df[target] <= upper_limit].copy()
```

실제 코드 15 / 23 · 원본 코드 열기 · 11_time_test_model.py

무엇을 구현했나

Q1~Q3 학습, Q4 평가. 학습셋의 99 백분위 초과만 제거

사용자에게 연결되는 결과

Q1~Q3 학습 후보 63,612 행

Q4 평가 후보 21,063 행

개발 관점의 설계 판단

이상치 기준은 학습셋에서만 계산합니다. 미래 평가 분기의 고매출 행은 제거하지 않습니다.

범주형 인코딩과 Random Forest

상권명 · 업종명을 모델이 처리할 수 있는 숫자로 변환

11_time_test_model.py · L111–L132

```
preprocessor = ColumnTransformer(
    transformers=[
        (
            "category",
            OneHotEncoder(
                handle_unknown="ignore"
            ),
            categorical_features
        )
    ],
    remainder="passthrough"
)
model = RandomForestRegressor(
    n_estimators=200,
    max_depth=20,
    min_samples_leaf=3,
    random_state=42,
    n_jobs=-1
)
```

실제 코드 16 / 23 · 원본 코드 열기 · 11_time_test_model.py

무엇을 구현했나

One-Hot 인코딩, 수치형 통과, 트리 200 개 · 깊이 20

사용자에게 연결되는 결과

새 범주도 오류 없이 처리하는 전처리 구성

개발 관점의 설계 판단

미관측 범주는 ignore 로 처리해 추론 오류를 줄입니다. 다만 새로운 상권의 예측 신뢰성이 보장되지는 않습니다.

전처리와 학습을 하나의 파이프라인으로

학습과 추론에 같은 전처리 순서를 적용

11_time_test_model.py · L136–L147

```
pipeline = Pipeline(
    steps=[
        (
            "preprocessor",
            preprocessor
        ),
        (
            "model",
            model
        )
    ]
)
```

실제 코드 17 / 23 · 원본 코드 열기 · 11_time_test_model.py

무엇을 구현했나

preprocessor → model 을 Pipeline 의 두 단계로 연결

사용자에게 연결되는 결과

fit 시 인코더와 모델을 함께 학습 , predict 시 같은 변환 재사용

개발 관점의 설계 판단

인코더와 모델을 하나로 저장해 학습 · 서비스의 변환 순서를 일치시킵니다 . 입력 컬럼 계약을 유지하는 구조입니다 .

MAE·RMSE·R^2 계산

오차 크기와 설명력을 함께 평가

11_time_test_model.py · L162–L181

```
pred = pipeline.predict(
    X_test
)
mae = mean_absolute_error(
    y_test,
    pred
)
rmse = mean_squared_error(
    y_test,
    pred
) ** 0.5
r2 = r2_score(
    y_test,
    pred
```

실제 코드 18 / 23 · 원본 코드 열기 · 11_time_test_model.py

무엇을 구현했나

Q4 X_test 예측 후 실제 y_test 와 세 지표 비교

사용자에게 연결되는 결과

MAE: 평균 절대오차 / RMSE: 큰 오차에 민감 / R^2: 설명력

개발 관점의 설계 판단

단일 지표 대신 오차 크기와 설명력을 함께 봅니다 . 같은 분기 실측 입력을 쓰므로 엄밀한 사전 미래 예측과 구분합니다 .

평가 수치는 표본과 실행 조건을 함께 공개

기존 저장 예측 파일의 실제값 · 예측값을 재계산했습니다 .

MAE

1,470 만원

평균 절대오차
정확값 14,704,847 원

RMSE

2,580 만원

큰 오차에 민감
정확값 25,795,030 원

R^2

0.5227

평가 표본 변동의 약 52.3% 설명
정확도 52.27% 를 뜻하지 않음

기존 저장 결과 20,844 행 ≠ 현재 Q4 21,063 행 . 현재 코드로 재평가한 성능은 아닙니다 .

최종 학습 모델 저장

웹 요청마다 모델을 새로 학습하지 않도록 결과를 보관

12_final_ai_model.py · L118–L130

```
pipeline.fit(
    X,
    y
)
print(" 최종  AI  모델 학습 완료 !")
joblib.dump(
    pipeline,
    BASE_DIR / 'models/final_sales_model.pkl'
)
```

실제 코드 19 / 23 · 원본 코드 열기 · 12_final_ai_model.py

무엇을 구현했나

최종 학습된 Pipeline 을 joblib.dump 로 저장

사용자에게 연결되는 결과

models/final_sales_model.pkl 을 서비스에서 재사용

개발 관점의 설계 판단

서비스는 저장된 Pipeline 을 읽습니다 . 전체 데이터로 학습한 최종 모델 저장은 별도의 성능 평가가 아닙니다 .

Streamlit 데이터 · 모델 캐시

화면 재실행마다 CSV 읽기와 모델 로딩이 반복되는 부담 감소

app.py · L26–L36

```python
@st.cache_data
def load_data():
    return (
        pd.read_csv(BASE_DIR / "data/processed/startup_ranking_map.csv"),
        pd.read_csv(BASE_DIR / "data/processed/hotplace_score_map.csv"),
    )
@st.cache_resource
def load_model():
    return joblib.load(BASE_DIR / "models/final_sales_model.pkl")
```

실제 코드 20 / 23 · 원본 코드 열기 · app.py

무엇을 구현했나

cache_data 로 데이터 반환값 , cache_resource 로 모델 인스턴스 재사용

사용자에게 연결되는 결과

사전 계산된 추천 CSV 와 최종 pkl 을 화면에 연결

개발 관점의 설계 판단

CSV 반환값은 cache_data, 모델 객체는 cache_resource 로 관리합니다 . 화면 재실행과 무거운 자원 로딩의 책임을 구분했습니다 .

선택 업종의 TOP10 후보 추출

사용자 입력을 실제 추천 테이블 조건으로 연결

app.py · L194–L214

```
result = startup[
    startup[" 서비스 _ 업종 _ 코드 _ 명 "]
    == selected_industry
].copy()
result = result.sort_values(
    " 평균 _ 창업추천점수 ",
    ascending=False
)
result = result.head(10)
result = result.reset_index(drop=True)
result[" 순위 "] = result.index + 1
```

실제 코드 21 / 23 · 원본 코드 열기 · app.py

무엇을 구현했나

선택한 업종 필터 → 평균 추천점수 내림차순 → head(10)

사용자에게 연결되는 결과

사용자 업종 선택 → TOP10

규칙 점수 기준 순위 표시

개발 관점의 설계 판단

데이터 필터 · 정렬과 모델 추론을 분리했습니다 . 사용자가 선택한 조건을 설명 가능한 결과로 연결합니다 .

화면 입력을 모델 컬럼에 맞춰 예측

평균 집계 테이블과 학습 입력의 컬럼명을 일치시킴

app.py · L237–L250

```
ai_input = ai_input.rename(
    columns={
        " 평균 _ 총 _ 유동인구 ": " 총 _ 유동인구 _ 수 ",
        " 평균 _2030 비율 ": "2030 비율 ",
        " 평균 _ 점포 _ 수 ": " 점포 _ 수 ",
        " 평균 _ 폐업비율 ": " 폐업비율 "
    }
)
result["AI_ 예상 _ 월매출 "] = ai_model.predict(
    ai_input
)
```

실제 코드 22 / 23 · 원본 코드 열기 · app.py

무엇을 구현했나

평균 _* 컬럼을 학습 컬럼으로 변경 후 predict(ai_input)

사용자에게 연결되는 결과

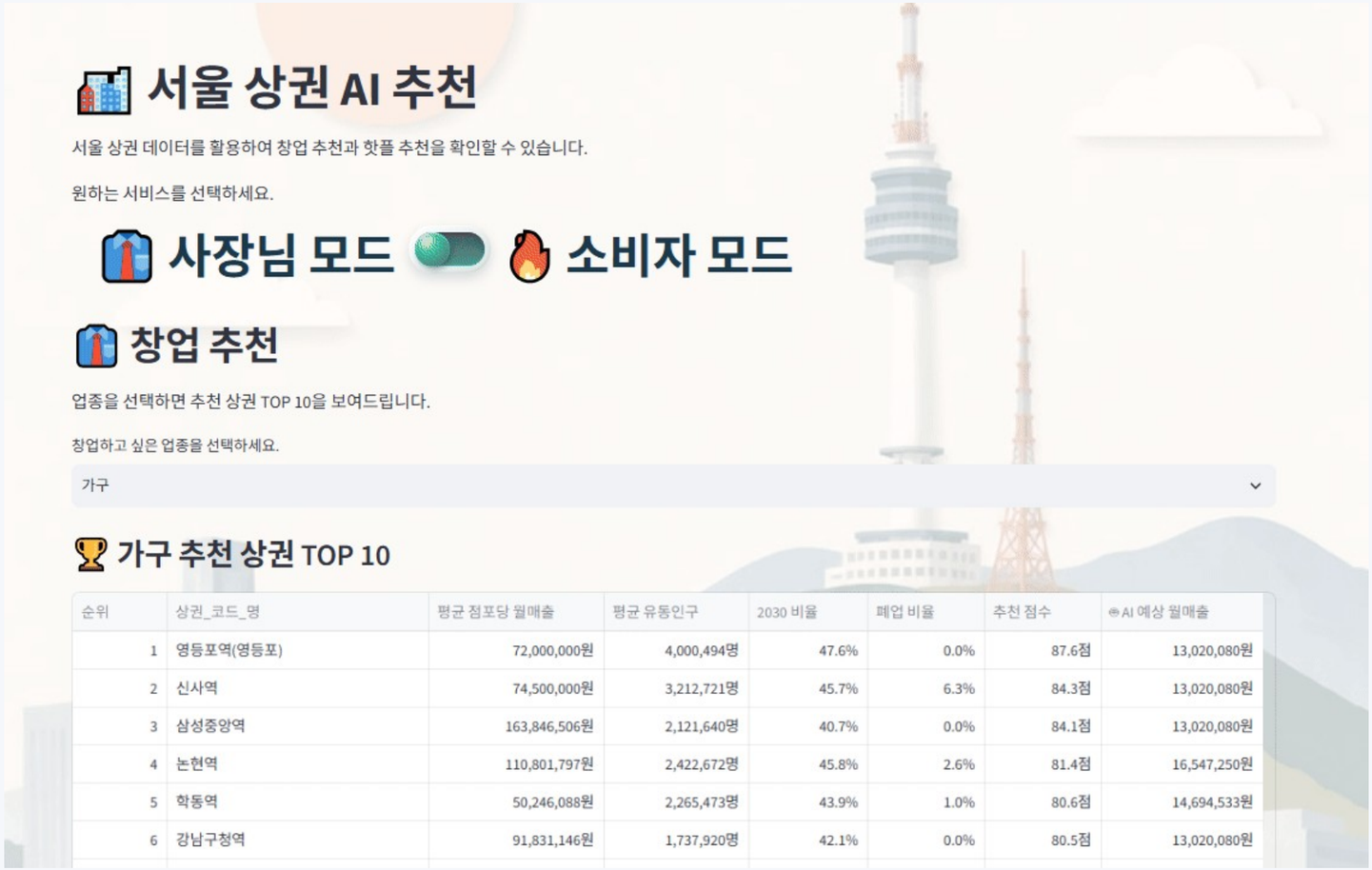
AI_ 예상 _ 월매출 컬럼에 예측 결과 추가

개발 관점의 설계 판단

평균 집계 컬럼을 학습 컬럼명으로 맞춰 추론합니다 . 평균 입력 예측과 분기별 예측 평균은 다를 수 있습니다 .

분석 지표 · 추천 순위 · 예상 매출을 한 화면에

첨부 자료의 실제 서비스 캡처를 사용했습니다 .



순위	상권_코드_명	평균 점포당 월매출	평균 유동인구	2030 비율	폐업 비율	추천 점수	AI 예상 월매출
1	영등포역(영등포)	72,000,000원	4,000,494명	47.6%	0.0%	87.6점	13,020,080원
2	신사역	74,500,000원	3,212,721명	45.7%	6.3%	84.3점	13,020,080원
3	삼성중앙역	163,846,506원	2,121,640명	40.7%	0.0%	84.1점	13,020,080원
4	논현역	110,801,797원	2,422,672명	45.8%	2.6%	81.4점	16,547,250원
5	학동역	50,246,088원	2,265,473명	43.9%	1.0%	80.6점	14,694,533원
6	강남구청역	91,831,146원	1,737,920명	42.1%	0.0%	80.5점	13,020,080원

사용자 흐름

① 업종을 선택한다

② 추천 TOP10 을 비교한다

③ 원래 지표와 예상 매출을 본다

④ 지도에서 위치를 확인한다

캡처의 예측값은 과거 화면 예시이며
신규 검증 결과가 아닙니다 .

사용자 질문을 키워드 조회로 처리

질문에서 목적을 판별해 해당 지표 순위를 조회

app.py · L538–L551

```
question = "".join(consumer_question.split())
if (
    " 주말 " in question
    and
    (" 유동 " in question or " 사람 " in question)
):
    weekend_result = latest.sort_values(
        " 주말 _ 유동인구 _ 수 ",
        ascending=False
    ).head(5)
```

실제 코드 23 / 23 · 원본 코드 열기 · app.py

무엇을 구현했나

공백 제거 후 주말 · 유동 · 사람 키워드 검사 , 주말 인구 상위 5 개

사용자에게 연결되는 결과

저장된 최신 분기 데이터에서 질문에 맞는 상권 반환

개발 관점의 설계 판단

질문을 공백 정규화한 뒤 키워드로 분기합니다 . 생성형 AI 호출이 아닌 데이터 조회라는 구현 범위를 명확히 설명합니다 .

개발 역량을 보여주는 구현 근거

코드에서 확인되는 설계 판단을 기능과 연결했습니다 .

DATA CONTRACT

정합성 검증

명시적 병합 키
many_to_one 검증
분모 0· 핵심 결측 처리

REUSE

책임 분리

원본 · 가공 · 모델 경로 분리
공통 지도 · 차트 함수
전처리 + 모델 Pipeline

OPERATIONS

실행 비용 관리

사전 계산 CSV 로딩
데이터 · 모델 캐시 구분
학습과 화면 추론 분리

개인 기여 비율 · 성능 개선율처럼 자료로 확인되지 않은 주장은 추가하지 않았습니다 .

검증한 사실과 남은 과제를 구분한다

구현의 신뢰성은 수치와 한계를 함께 설명할 때 드러납니다 .

확인한 사실	핵심 결측 1,057 행 제외 · 정제 결과 결측 · 행 중복 0 · 지도 좌표 결측 0
평가 범위	기존 평가 CSV 재계산 · 현재 전체 Q4 재학습 · 재평가는 미실행
경제성 한계	임대료 · 권리금 · 인건비 · 원가 미포함 → 높은 매출이 높은 이익을 보장하지 않음
다음 검증	가중치 ±10% · 업종별 오차 · 관측 분기 수 · 반복 분기 검증

다음 고도화는 근거를 더 잘 남기는 것부터

구현 완료 항목과 향후 계획을 구분했습니다 .

01 QUALITY

출처 · 품질 로그

개별 URL· 수집일 · 단위 기록
제외 사유 · 병합 일치율
관측 분기 수 공개

02 VALIDATION

점수 · 모델 검증

가중치 민감도
업종별 기준모델 비교
전체 Q4 재평가

03 EXPANSION

범위와 갱신

2024~2026 공통 범위 구성
임대료 · 시설 품질 확인
정기 갱신과 롤링 검증

세 자료의 역할과 실제 근거

첨부자료의 내용은 참고 정보로 사용하고 프로젝트 코드와 대조했습니다 .

발표자료 PPT

내용과 서비스 화면

데이터 규모 · 분석 범위 · 처리 기준 · 평가와 한계

첨부된 스크린샷 ·GIF 발췌

핵심 코드 PDF

23 개 코드 해설

모든 코드 주제를 해당 분석 단계에 배치

파일 · 행 번호 · 원본 링크 유지

컬러 리뉴얼 PDF

가독성과 구성

진한 제목 · 밝은 본문

블루 강조 · 충분한 여백

핵심 수치 · 차트로 전달

소스 기준 b7bc901 · data/raw·processed CSV 실측 · q4_prediction_result.csv 재계산 . 코드 전문과 추가 설명은 발표 노트에 수록했습니다 .